Computer Programming: Lecture 8

Outline

- List
- Dictionaries

List

A list is a built-in data type in Python used to store collections of items.

A List can contain elements of different data types including other lists.

```
my_list = [1,2,3,4]
names =
[“Abebe”,”Bekele”,”Ayalew”]
```

Lists

Characteristics of lists:

- Lists can be indexed
- Lists are mutable
- List can contain heterogeneous collection of items

```
name = [“Abebe”,1,1.2]
```

List

Indexing - elements of list have indexes starting from 0 to the length of the list minus 1.

>>> my_list = [1,2,3,4]

>>> my_list[0] => 1

>>> my_list[1] = > 2

>>> my_list[2] => 3

>>> my_list[3] => 4

List

Negative indexes: you can also indexes lists with negative indexes. The index of the last element is -1, the second last is -2 and so on.

```
>>> my_list = [1,2,3,4]
>>> my_list[-1] => 4
>>> my_list[-2] => 3
>>> my_list[-3] => 2
>>> my_list[-4] => 1
```

List

Slicing: like strings you can also slice lists with indexes.

Syntax:

- mylist[start index: end index: step]
  - The start index default is 0
  - The end index default is length of the list len(list) - 1
  - The step default is 1
  - Slicing retrieves elements from the start index to end index - 1

List

Slicing examples:

```
>>> my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> my_list[0:5] => [0, 1, 2, 3, 4]
>>> my_list[:5] => [0, 1, 2, 3, 4]
>>> my_list[5:] => [5, 6, 7, 8, 9, 10]
>>> my_list[::2] => [0, 2, 4, 6, 8, 10]
>>> my_list[::-1] => [10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
>>> my_list[8:0:-2] =>[8, 6, 4, 2]
>>> my_list[8::-2] => [8, 6, 4, 2,0]
```

Lists

Modifying lists - lists are mutable and the elements of a list can be modified.

```
>>> my_list = [1,2,3,4,5]
>>> my_list[1] = 6
>>> my_list => [1,6,3,4,5]
```

List Operations

Adding element to a list:

- Append - add a single element at the end of a list
- Insert - adds an element to a specific index.
- Extend - Add multiple elements at the end of a list.

```
>>> my_list = [1,2,3,4,5]

>>> my_list.append(6)

>>> my_list => [1, 2, 3, 4, 5, 6]
```

List Operations

Insert

```
>>> my_list = [1,2,3,4,5]
>>> my_list.insert(1,0) #the first argument is the index and the second is an item
>>> my_list => [1,0,2,3,4,5]
```

Extend

```
>>> my_list = [1,2,3,4,5]
>>> my_list.extend([6,7,8])
>>> my_list => [1, 2, 3, 4, 5, 6, 7, 8]
```

List Operations

Removing an element:

- Pop - removes the last element if called with no argument or removes an element at a specific index.
- Remove - called by passing an element as an argument and it removes the first occurrence of the element.
- Clear - removes all elements from the list.

List Operations

```
>>> my_list = [1,2,3,4,5]
>>> my_list.pop()
>>> my_list => [1, 2, 3, 4]
>>> my_list.pop(0)
>>> my_list => [2,3,4]
>>> my_list.remove(2)
>>> my_list => [3,4]
>>> my_list.clear()
>>> my_list => []
```

List Operations

Concatenation and repetition:

- + - operator can be used to concatenate two lists
- * - operator can be used to repeat lists.

>>> [1,2,3] + [4,5,6] => [1,2,3,4,5,6]

>>> [1,2,3] * 2 => [1,2,3,1,2,3]

Traversing lists

```
i = 0
my_list = [1,2,3,4,5]
while(i < len(my_list)):
    print(my_list[i])
    i+=1
```

```
my_list = [1,2,3,4,5]
for i in my_list:
    print(i)
```

Nested List

We can create a list containing other lists.

>>> my_list = [[1,2,3],[3,4,5]]

>>> my_list[1] => [3, 4, 5]

>>> my_list[1][1] => 4